Pixel Neural Networks

Jordy Saltos, Juan Carlos Pajares

February 2026

Contents

1 Introduction **2**

2 PixelRNN: Recurrent Architectures **2**
2.1 Row LSTM 2
2.2 Diagonal BiLSTM 3

3 Training Techniques **3**

4 PixelCNN and Extensions **4**
4.1 Gated PixelCNN 4
4.2 Conditional PixelCNN 5

Appendix: Figures **6**

1 Introduction

Autoregressive models for image generation aim to model the joint distribution of all pixels by factorizing it as a product of conditional distributions:

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p(x_i \mid x_{<i}), \tag{1}$$

where each pixel x_i is predicted conditioned on all previously generated pixels $x_{<i}$ in raster scan order. This framework, introduced by *PixelRNN* and *PixelCNN*, enables explicit density estimation and high-quality image synthesis.

A digital image is represented as a three-dimensional matrix of dimensions $3 \times N_{\text{row}} \times N_{\text{col}}$, encoding the RGB intensity of each pixel in the discrete range $[0, 255]$. For color images, the conditional distribution at each pixel is further decomposed channel by channel:

$$p(x_i \mid x_{<i}) = p(x_{i,R} \mid x_{<i})\, p(x_{i,G} \mid x_{<i}, x_{i,R})\, p(x_{i,B} \mid x_{<i}, x_{i,R}, x_{i,G}), \tag{2}$$

capturing both spatial and inter-channel dependencies.

2 PixelRNN: Recurrent Architectures

PixelRNN adapts the LSTM structure to the two-dimensional image domain by replacing linear transformations with convolutions. The general two-dimensional LSTM dynamics are (see Figure 3):

$$\begin{cases} I_i = \sigma(\mathbf{X}_i W_{xi} + H_{i-1} W_{hi} + \mathbf{b}_i)\,, \\ F_i = \sigma(\mathbf{X}_i W_{xf} + H_{i-1} W_{hf} + \mathbf{b}_f)\,, \\ O_i = \sigma(\mathbf{X}_i W_{xo} + H_{i-1} W_{ho} + \mathbf{b}_o)\,, \\ \tilde{C}_i = \tanh(\mathbf{X}_i W_{x\tilde{c}} + H_{i-1} W_{h\tilde{c}} + \mathbf{b}_{\tilde{c}})\,, \\ C_i = F_i \odot C_{i-1} + I_i \odot \tilde{C}_i, \\ H_i = O_i \odot \tanh(C_i), \end{cases} \tag{3}$$

where I_i, F_i, O_i are the input, forget, and output gates; $\tilde{C}_i$ is the candidate cell state; and each gate has its own weight matrices $W_{x\cdot}$, $W_{h\cdot}$ and bias $\mathbf{b}_\cdot$.

2.1 Row LSTM

The Row LSTM treats each image row as a recurrent step (see Figure 4). Input and state contributions are computed via horizontal convolutions of size $1 \times k$:

$$
\begin{cases}
I_i = \sigma\big(K_{is}^{I} * \mathbf{X}_i + K_{ss}^{I} * H_{i-1} + \mathbf{b}_i\big)\,, \\
F_i = \sigma\big(K_{is}^{F} * \mathbf{X}_i + K_{ss}^{F} * H_{i-1} + \mathbf{b}_f\big)\,, \\
O_i = \sigma\big(K_{is}^{O} * \mathbf{X}_i + K_{ss}^{O} * H_{i-1} + \mathbf{b}_o\big)\,, \\
\tilde{C}_i = \tanh\Big(K_{is}^{\tilde{C}} * \mathbf{X}_i + K_{ss}^{\tilde{C}} * H_{i-1} + \mathbf{b}_{\tilde{c}}\Big)\,, \\
C_i = F_i \odot C_{i-1} + I_i \odot \tilde{C}_i, \\
H_i = O_i \odot \tanh(C_i),
\end{cases}
\tag{4}
$$

where $K_{is}^{(\cdot)}$ is a masked horizontal kernel applied to input row $\mathbf{X}_i$, and $K_{ss}^{(\cdot)}$ is a horizontal kernel applied to the previous hidden state H_{i-1}. All pixels in a row are updated simultaneously. The receptive field is triangular, covering rows above and pixels to the left in the current row.

2.2 Diagonal BiLSTM

The Diagonal BiLSTM achieves a complete global receptive field by processing the image along its diagonals (see Figure 6). To enable efficient diagonal convolutions, a *skewing* transformation is applied: each row is shifted one position to the right relative to the previous one, transforming an $n \times n$ image into an $n \times (2n-1)$ map $\tilde{\mathbf{X}}$ where diagonals become vertical columns (see Figure 5).

Gates are then computed column by column over the skewed map:

$$
\begin{cases}
I_i = \sigma\Big(K_{is}^{I} * \tilde{\mathbf{X}}_i + K_{ss}^{I} * \tilde{H}_{i-1} + \mathbf{b}_i\Big)\,, \\
F_i = \sigma\Big(K_{is}^{F} * \tilde{\mathbf{X}}_i + K_{ss}^{F} * \tilde{H}_{i-1} + \mathbf{b}_f\Big)\,, \\
O_i = \sigma\Big(K_{is}^{O} * \tilde{\mathbf{X}}_i + K_{ss}^{O} * \tilde{H}_{i-1} + \mathbf{b}_o\Big)\,, \\
\tilde{C}_i = \tanh\Big(K_{is}^{\tilde{C}} * \tilde{\mathbf{X}}_i + K_{ss}^{\tilde{C}} * \tilde{H}_{i-1} + \mathbf{b}_{\tilde{c}}\Big)\,, \\
C_i = F_i \odot C_{i-1} + I_i \odot \tilde{C}_i, \\
H_i = O_i \odot \tanh(C_i),
\end{cases}
\tag{5}
$$

where $K_{is}^{(\cdot)}$ is a 1×1 convolution and $K_{ss}^{(\cdot)}$ is a vertical 2×1 kernel. The architecture is bidirectional: diagonals are processed in both directions and outputs are combined. After undoing the skewing, the result is a feature map H of the same size as the original image with full global context.

3 Training Techniques

Two key techniques enable stable and effective training of deep PixelRNNs :

Residual connections — Each LSTM layer is wrapped in a residual block that adds its input to its output via 1×1 convolutions. This prevents gradient vanishing and allows stacking up to twelve LSTM layers without performance degradation. (see Figure 7)

Masked convolutions — To enforce causality, convolutional filters are masked to zero out connections to future pixels or channels. This guarantees that each prediction depends only on valid past context, respecting the autoregressive factorization (see Figure 8).

4 PixelCNN and Extensions

PixelCNN replaces recurrent layers with masked convolutions entirely, enabling full spatial parallelization during training at the cost of a bounded receptive field. The output of layer k is:

$$h^{(k)} = \phi(W_k * h^{(k-1)}), \tag{6}$$

where W_k is a masked filter and $\phi(\cdot)$ is a pointwise activation. This makes PixelCNN significantly faster to train than PixelRNN, though slightly less expressive (see Figure 9).

4.1 Gated PixelCNN

To recover expressive power comparable to PixelRNN, the Gated PixelCNN replaces standard activations with a multiplicative gating unit:

$$h^{(k)} = \tanh\Big(W_{k,f} * h^{(k-1)}\Big) \odot \sigma\Big(W_{k,g} * h^{(k-1)}\Big), \tag{7}$$

where $W_{k,f}$ and $W_{k,g}$ are distinct filters. The tanh branch produces a candidate representation and the σ branch acts as a dynamic gate, introducing multiplicative interactions that increase the model's capacity to capture complex and multimodal distributions (see Figure 10).

To eliminate the *blind spot* present in standard masked convolutions, the architecture splits into two parallel stacks:

- **Vertical stack** — conditions on all rows above the current pixel.
- **Horizontal stack** — conditions on all pixels to the left in the current row.

The outputs of both stacks are fused at each layer, yielding a fully rectangular receptive field while preserving the autoregressive property:

$$p(x) = \prod_{i=1}^{n^2} p(x_i \mid x_{<i}). \tag{8}$$

4.2 Conditional PixelCNN

The Conditional PixelCNN extends the model to conditional generation by incorporating a high-level latent variable H :

$$p(x \mid H) = \prod_{i=1}^{n^2} p(x_i \mid x_{<i}, H). \tag{9}$$

Global conditioning — H is injected as an additional bias in each gated layer:

$$h^{(k)} = \tanh\Big(W_{k,f} * h^{(k-1)} + V_{k,f}^T H\Big) \odot \sigma\Big(W_{k,g} * h^{(k-1)} + V_{k,g}^T H\Big). \tag{10}$$

Spatial conditioning — When H carries spatial structure, it is first upsampled via a deconvolutional network $s = m(H)$, and then injected as a spatially-varying bias:

$$h^{(k)} = \tanh\Big(W_{k,f} * h^{(k-1)} + V_{k,f} * s\Big) \odot \sigma\Big(W_{k,g} * h^{(k-1)} + V_{k,g} * s\Big). \tag{11}$$

This allows location-dependent conditioning and enables controlled generation of images consistent with specific structural constraints.

Appendix: Figures

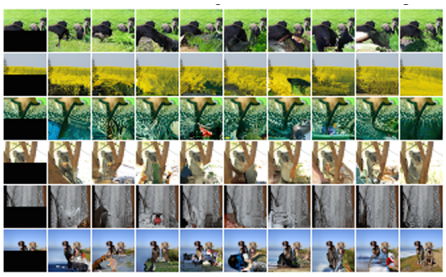

Figure 1: Example of generated images. First column: partially hidden image. Last column: real image. Remaining columns: generated images. [1]

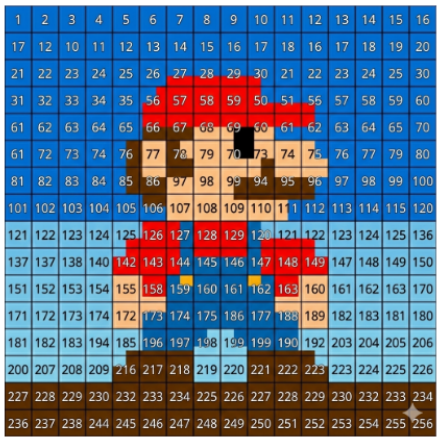

Figure 2: Example of an image represented as a matrix of pixels. [5]

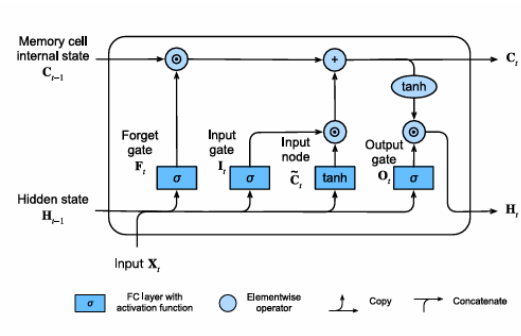

Figure 3: Classical LSTM architecture. [3]

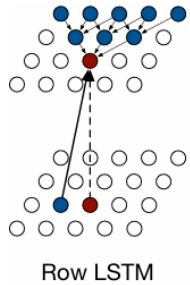

Figure 4: Row LSTM receptive field. [1]

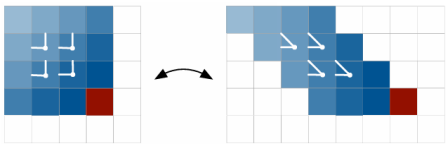

Figure 5: Application of *skewing* to an image. [1]

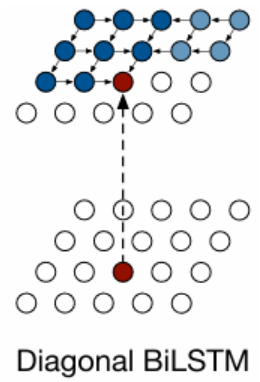

Figure 6: Input-to-state and state-to-state mappings for the Diagonal BiLSTM architecture. [1]

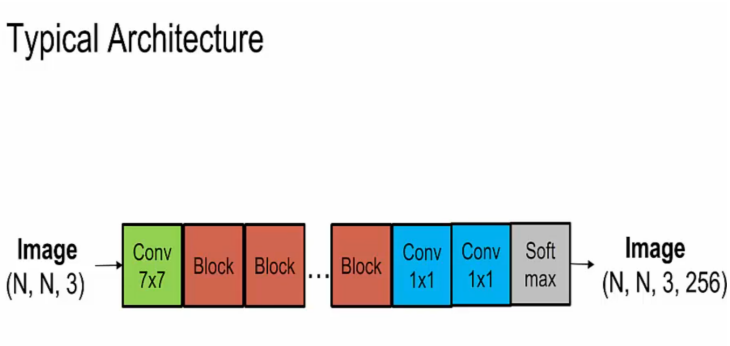

Figure 7: Typical deep architecture with residual blocks. [4]

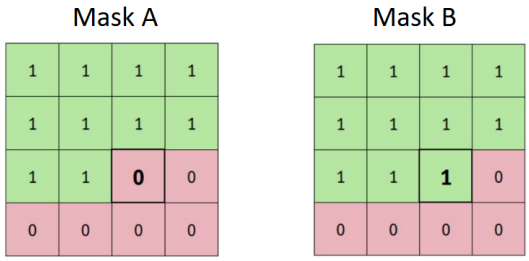

Figure 8: Difference between Mask A and Mask B. [1]

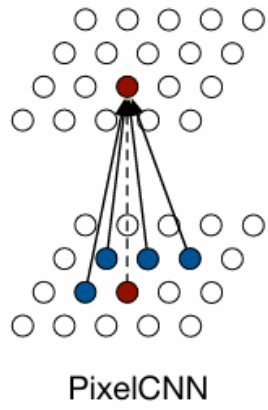

Figure 9: PixelCNN architecture overview. [1]

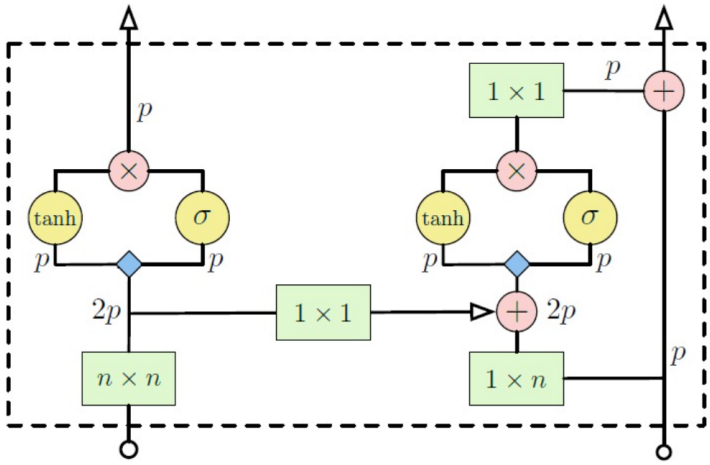

Figure 10: Gated PixelCNN architecture.[2]

References

[1] van den Oord, A., Kalchbrenner, N., and Kavukcuoglu, K. *Pixel Recurrent Neural Networks*. Proceedings of the 33rd International Conference on Machine Learning (ICML), New York, NY, USA, 2016. JMLR: W&CP volume 48.

[2] van den Oord, A., Kalchbrenner, N., Vinyals, O., Espeholt, L., Graves, A., and Kavukcuoglu, K. *Conditional Image Generation with PixelCNN Decoders*. Advances in Neural Information Processing Systems (NeurIPS), arXiv preprint arXiv:1606.05328, 2016.

[3] Amara-Ouali, Y. *Introduction Deep Learning* [Online]. Available at: https://www.yvenn-amara.com. Accessed: March 2026.

[4] Dong, H. *Lecture 6: Autoregressive Models*. Lecture Notes, Peking University. s.f.

[5] Google DeepMind. *AI-generated image* [Image generated by artificial intelligence]. Google, 2025. Generated using the Gemini model. Available at: https://gemini.google.com.